অধ্যায় ৬

Arrays এবং Data Processing

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Array তৈরি করতে এবং Index দিয়ে Element পড়তে ও বদলাতে।
- `push`, `pop`, `shift` ও `unshift` দিয়ে Element যোগ ও বাদ দিতে।
- `slice` এবং `splice`-এর পার্থক্য ব্যাখ্যা করতে।
- `for...of` দিয়ে Array-র প্রতিটি মান পড়তে।
- `map`, `filter` ও `find` দিয়ে Data রূপান্তর, বাছাই ও খোঁজার কাজ করতে।
- `reduce` দিয়ে একটি তালিকা থেকে একটি সারমান বের করতে।
- কোন Method মূল Array বদলায় এবং কোনটি নতুন Array দেয়, তা চিনতে।
- একাধিক Method জুড়ে বাস্তব Data রূপান্তর করতে।
- একটি সম্পূর্ণ **Inventory Dashboard** তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৫ সম্পন্ন করা: Arrow Function, Callback এবং Pure Function।
- অধ্যায় ৪-এর `for` Loop এবং Array-র প্রাথমিক পরিচয় (Index, `length`)।
- অধ্যায় ২-এর Reference: Object ও Array-তে ঠিকানা Copy হয়, মান নয়।
- `chapter-06` নামে একটি নতুন Folder।

এই অধ্যায় কেন গুরুত্বপূর্ণ

প্রায় সব Program-এর কেন্দ্রে থাকে তালিকা: পণ্যের তালিকা, শিক্ষার্থীর তালিকা, বার্তার তালিকা, লেনদেনের তালিকা। অধ্যায় ৪-এ আপনি Loop দিয়ে তালিকা ঘুরে যোগফল, গণনা ও সর্বোচ্চ বের করেছেন। সেটি কাজ করে, কিন্তু প্রতিবার Counter, সীমা এবং Accumulator নিজে সামলাতে হয়, আর সেখানেই Off-by-One Error ঢোকে।

JavaScript-এর Array এই সাধারণ কাজগুলোর জন্য তৈরি Method দেয়। "প্রতিটি দামে কর যোগ করো", "যেগুলোর মজুত আছে শুধু সেগুলো রাখো", "সব মিলিয়ে মোট বের করো"—প্রতিটির জন্য একটি করে Method আছে, এবং Code পড়লেই উদ্দেশ্য বোঝা যায়।

এই অধ্যায়ের দ্বিতীয় বড় বিষয় **Mutation**: কোন কাজ মূল তালিকাটিকে বদলে ফেলে, আর কোনটি নতুন তালিকা তৈরি করে। এই পার্থক্য না জানলে একটি তালিকা "অজান্তে" বদলে যাওয়ার Bug বারবার দেখা দেয়।

৬.১ Array কেন প্রয়োজন

তিনটি পণ্যের দাম রাখতে তিনটি Variable লাগে: `price1`, `price2`, `price3`। তিনশটি হলে? আর সংখ্যাটি যদি আগে থেকে জানা না থাকে?

Array একটি Variable-এ ক্রমানুসারে যেকোনো সংখ্যক মান রাখে। তালিকা বড়-ছোট হতে পারে, এবং পুরো তালিকাটি একটি নাম দিয়ে Function-এ পাঠানো যায়।

why-arrays.js · JavaScript — চলবে: Node.js

```
const prices = [120, 15, 950];

console.log(prices);
console.log(prices.length);
console.log(typeof prices);
console.log(Array.isArray(prices));
```

Output

```
[ 120, 15, 950 ]
3
object
true
```

Array আসলে একটি বিশেষ ধরনের Object, তাই `typeof` দেখায় `object`। কোনো মান Array কি না জানতে `Array.isArray()` ব্যবহার করুন।

> **নোট:** `console.log()` একটি Array-কে কীভাবে দেখাবে, তা পরিবেশভেদে সামান্য আলাদা। এই অধ্যায়ের Output Node.js-এর; Browser Console একই তথ্য একটু ভিন্ন সাজে দেখায়। তাই Array সরাসরি দেখানো উদাহরণগুলো Node.js-এ চালান।

৬.২ Array তৈরি করা

সবচেয়ে প্রচলিত উপায় **Array Literal**: তৃতীয় বন্ধনীর ভেতরে কমা দিয়ে আলাদা করা মান।

creating.js · JavaScript — চলবে: Node.js

```
const empty = [];
const fruits = ['আম', 'কলা', 'লিচু'];
const mixed = [42, 'লেখা', true, null];
const product = [{ name: 'Pen', price: 15 }];

console.log(empty);
console.log(fruits);
console.log(mixed);
console.log(product[0].name);
```

Output

```
[]
[ 'আম', 'কলা', 'লিচু' ]
[ 42, 'লেখা', true, null ]
Pen
```

- Array ফাঁকা হতে পারে; পরে তাতে Element যোগ করা যায়।
- একই Array-তে ভিন্ন Type-এর মান রাখা সম্ভব, কিন্তু উচিত নয়। একটি Array-তে একই ধরনের জিনিস রাখুন: সব দাম, অথবা সব পণ্য।
- Element একটি Object-ও হতে পারে। `product[0]` Object-টি দেয়, তারপর `.name` তার তথ্য পড়ে।

Array-র নাম বহুবচনে রাখুন (`prices`, `students`), আর একটি Element-এর নাম একবচনে (`price`, `student`)। তাহলে Code পড়েই বোঝা যায় কোনটি তালিকা আর কোনটি একটি মান।

৬.৩ Index দিয়ে পড়া ও বদলানো

অধ্যায় ৪ থেকে মনে করুন: Index শূন্য থেকে শুরু, এবং শেষ Index হলো `length - 1`।

indexing.js · JavaScript চলবে: Node.js

```
const fruits = ['আম', 'কলা', 'লিচু'];

console.log(fruits[0]);
console.log(fruits[fruits.length - 1]);
console.log(fruits.at(-1));
console.log(fruits[5]);

fruits[1] = 'কাঁঠাল';
console.log(fruits);

console.log(fruits.includes('লিচু'));
console.log(fruits.indexOf('লিচু'));
console.log(fruits.indexOf('জাম'));
```

Output

```
আম
লিচু
লিচু
undefined
[ 'আম', 'কাঁঠাল', 'লিচু' ]
true
2
-1
```

- **শেষ Element** পাওয়ার দুটি উপায়: `fruits[fruits.length - 1]`, অথবা সংক্ষেপে `fruits.at(-1)`। `at()`-এ ঋণাত্মক সংখ্যা শেষ থেকে গোনে।
- **সীমার বাইরের Index** Error দেয় না; `undefined` দেয়।
- **Index-এ Assignment** সেই জায়গার মান বদলে দেয়। `fruits` একটি `const`, তবু এটি সম্ভব, কারণ `const` শুধু নামটিকে একই Array-র সঙ্গে বেঁধে রাখে; Array-র ভেতরের মান বদলানো আটকায় না (অধ্যায় ২)।
- **`includes()`** জানায় মানটি আছে কি না। **`indexOf()`** তার Index দেয়, আর না পেলে `-1`।

> **সতর্কতা:** `length`-এর চেয়ে বড় Index-এ মান রাখবেন না (যেমন তিন Element-এর Array-তে `fruits[10] = 'জাম'`)। JavaScript মাঝের জায়গাগুলো ফাঁকা রেখে Array-টিকে এগারো Element-এর বানিয়ে দেবে। শেষে যোগ করতে `push()` ব্যবহার করুন।

৬.৪ Element যোগ ও বাদ: push, pop, shift, unshift

Array-র দুই প্রান্তে Element যোগ ও বাদ দেওয়ার চারটি Method আছে।

Method	কাজ	কোন প্রান্তে	ফেরত দেয়
`push(value)`	যোগ করে	শেষে	নতুন `length`
`pop()`	বাদ দেয়	শেষ থেকে	বাদ পড়া Element
`unshift(value)`	যোগ করে	শুরুতে	নতুন `length`
`shift()`	বাদ দেয়	শুরু থেকে	বাদ পড়া Element

push-pop.js · JavaScript — চলবে: Node.js

```
const queue = ['Sara', 'Rafi'];

const newLength = queue.push('Mina');
console.log(newLength);
console.log(queue);

const last = queue.pop();
console.log(last);

queue.unshift('Nila');
console.log(queue);

const first = queue.shift();
console.log(first);
console.log(queue);
```

Output

```
3
[ 'Sara', 'Rafi', 'Mina' ]
Mina
[ 'Nila', 'Sara', 'Rafi' ]
Nila
[ 'Sara', 'Rafi' ]
```

চারটিই **মূল Array-টিকে বদলে ফেলে**। `queue.push('Mina')`-এর পর `queue` আর আগের মতো নেই। এ ধরনের পরিবর্তনকে **Mutation** বলে, এবং বিষয়টি অনুচ্ছেদ ৬.১১-এ ফিরে আসবে।

ফেরত আসা মানটিও লক্ষ করুন: `push()` নতুন Array ফেরত দেয় না, দেয় নতুন `length`। তাই `const result = queue.push('X')` লিখে `result`-কে Array ভাবা একটি প্রচলিত ভুল।

`push` ও `pop` দ্রুত, কারণ শেষ প্রান্তে কাজ করলে বাকি Element-গুলো নিজের জায়গায় থাকে। `shift` ও `unshift`-এ প্রতিটি Element-এর Index এক ঘর সরে, তাই বড় Array-তে এগুলো তুলনামূলক ধীর।

৬.৫ slice এবং splice

নাম প্রায় এক, কাজ সম্পূর্ণ আলাদা। এই দুটির পার্থক্যই এই অধ্যায়ের মূল ধারণার সবচেয়ে পরিষ্কার উদাহরণ।

slice: একটি অংশের Copy

`slice(start, end)` মূল Array না বদলে তার একটি অংশ নতুন Array হিসেবে ফেরত দেয়। `start` Index থেকে শুরু, `end` Index-এর **আগে** শেষ।

slice.js · JavaScript — চলবে: Node.js

```
const letters = ['a', 'b', 'c', 'd', 'e'];

console.log(letters.slice(1, 3));
console.log(letters.slice(2));
console.log(letters.slice(-2));
console.log(letters.slice());
console.log(letters);
```

Output

```
[ 'b', 'c' ]
[ 'c', 'd', 'e' ]
[ 'd', 'e' ]
[ 'a', 'b', 'c', 'd', 'e' ]
[ 'a', 'b', 'c', 'd', 'e' ]
```

- `slice(1, 3)` — Index ১ ও ২; Index ৩ অন্তর্ভুক্ত নয়।
- `slice(2)` — Index ২ থেকে শেষ পর্যন্ত।
- `slice(-2)` — শেষ দুটি।
- `slice()` — পুরো Array-র একটি Copy।

শেষ লাইন দেখাচ্ছে `letters` অপরিবর্তিত।

splice: মূল Array কেটে বদলানো

`splice(start, deleteCount, ...items)` মূল Array-র `start` Index থেকে `deleteCount` টি Element সরিয়ে দেয়, এবং চাইলে সেই জায়গায় নতুন Element বসায়। এটি সরানো Element-গুলো ফেরত দেয়।

splice.js · JavaScript — চলবে: Node.js

```
const letters = ['a', 'b', 'c', 'd', 'e'];

const removed = letters.splice(1, 2);
console.log(removed);
console.log(letters);

letters.splice(1, 0, 'x');
console.log(letters);

letters.splice(0, 1, 'z');
console.log(letters);
```

Output

```
[ 'b', 'c' ]
[ 'a', 'd', 'e' ]
[ 'a', 'x', 'd', 'e' ]
[ 'z', 'x', 'd', 'e' ]
```

- `splice(1, 2)` — Index ১ থেকে দুটি সরানো হলো।
- `splice(1, 0, 'x')` — কিছু সরানো হয়নি (`0`); Index ১-এ `'x'` ঢোকানো হলো।
- `splice(0, 1, 'z')` — একটি সরিয়ে তার জায়গায় `'z'` বসানো হলো।

	slice	splice
মূল Array বদলায়	না	হ্যাঁ
দ্বিতীয় Argument	শেষ Index (অন্তর্ভুক্ত নয়)	কয়টি সরাতে হবে
ফেরত দেয়	অংশটির নতুন Array	সরানো Element-গুলো
ব্যবহার	অংশ পড়া, Copy করা	মাঝখান থেকে সরানো, ঢোকানো, বদলানো

মনে রাখার সহজ উপায়: `splice`-এ একটি অতিরিক্ত `p` আছে, এবং এটিই Array-টিকে **p**রিবর্তন করে।

৬.৬ for...of

Index-এর প্রয়োজন না থাকলে Array-র প্রতিটি মান পড়ার সবচেয়ে পরিষ্কার উপায় `for...of`।

for-of.js · JavaScript — চলবে: Browser Console ও Node.js

```
const names = ['Sara', 'Rafi', 'Mina'];

for (const name of names) {
  console.log(name);
}
```

Output

```
Sara
Rafi
Mina
```

প্রতিটি Iteration-এ `name` -এ পরের Element-টি আসে। Counter নেই, সীমা নেই, `names[i]` নেই; তাই Off-by-One Error-এর সুযোগও নেই। `break` ও `continue` এখানে স্বাভাবিকভাবেই কাজ করে।

for-of-break.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950, 280];
let firstExpensive = null;

for (const price of prices) {
  if (price > 500) {
    firstExpensive = price;
    break;
  }
}

console.log(firstExpensive);
```

Output

```
950
```

কখন	কোন Loop
শুধু মান দরকার	`for...of`
Index-ও দরকার, বা নির্দিষ্ট পরিসর	`for (let i = 0; ...)`
রূপান্তর, বাছাই বা সারমান	`map`, `filter`, `reduce` (নিচে)

> **সতর্কতা:** `for...in` নামে প্রায় একই চেহারার আরেকটি Loop আছে। সেটি মান নয়, Index (String হিসেবে) দেয়, এবং Object-এর জন্য তৈরি। Array-তে `for...in` ব্যবহার করবেন না।

৬.৭ map

`map` একটি Array-র প্রতিটি Element-কে রূপান্তর করে একই দৈর্ঘ্যের একটি **নতুন Array** তৈরি করে। আপনি একটি Callback দেন (অধ্যায় ৫), যা একটি Element নিয়ে তার নতুন রূপ ফেরত দেয়; `map` সেটি প্রতিটি Element-এর ওপর চালায়।

map.js · JavaScript — চলবে: Node.js

```
const prices = [120, 15, 950];

const withTax = prices.map((price) => price + Math.round(price * 15 / 100));
const labels = prices.map((price) => `${price} টাকা`);

console.log(withTax);
console.log(labels);
console.log(prices);
```

Output

```
[ 138, 17, 1093 ]
[ '120 টাকা', '15 টাকা', '950 টাকা' ]
[ 120, 15, 950 ]
```

একই কাজ `for` Loop দিয়ে করতে একটি ফাঁকা Array তৈরি করে প্রতিটি ফল `push` করতে হতো। `map` সেই ছাঁচটি নিজে সামলায়; আপনাকে শুধু বলতে হয় একটি Element-এর কী হবে।

Object-এর তালিকা থেকে একটি তথ্য বের করে আনা `map`-এর সবচেয়ে প্রচলিত ব্যবহার।

map-objects.js · JavaScript — চলবে: Node.js

```
const products = [
  { name: 'Notebook', price: 120 },
  { name: 'Pen', price: 15 },
];

const names = products.map((product) => product.name);

console.log(names);
console.log(names.join(', '));
```

Output

```
[ 'Notebook', 'Pen' ]
Notebook, Pen
```

`join()` একটি Array-র Element-গুলোকে মাঝখানে একটি String বসিয়ে জোড়া লাগায়। অধ্যায় ৪-এ নামের তালিকা বানাতে যে Ternary লিখতে হয়েছিল, `join(', ')` সেটি এক ধাপে করে।

`map`-এর দুটি নিয়ম:

- Callback-কে অবশ্যই একটি মান ফেরত দিতে হবে। `{ }` ব্যবহার করে `return` ভুলে গেলে ফলের Array `undefined`-এ ভরে যায়।
- ফলের দৈর্ঘ্য সব সময় মূলের সমান। কিছু বাদ দিতে হলে সেটি `filter`-এর কাজ।

৬.৮ filter

`filter` প্রতিটি Element-এর ওপর একটি শর্ত পরীক্ষা করে এবং যেগুলোতে শর্ত সত্য, শুধু সেগুলো নিয়ে একটি **নতুন Array** তৈরি করে। Callback-টি `true` বা `false` ফেরত দেয়।

filter.js · JavaScript — চলবে: Node.js

```
const prices = [120, 15, 950, 280];

console.log(prices.filter((price) => price >= 200));
console.log(prices.filter((price) => price > 5000));

const products = [
  { name: 'Notebook', stock: 40 },
  { name: 'Pen', stock: 0 },
  { name: 'Bag', stock: 5 },
];

const available = products.filter((product) => product.stock > 0);

console.log(available.length);
console.log(available.map((product) => product.name));
```

Output

```
[ 950, 280 ]
[]
2
[ 'Notebook', 'Bag' ]
```

- ফলের Array মূলের চেয়ে ছোট বা সমান হয়; Element-গুলো অপরিবর্তিত থাকে এবং ক্রম বজায় থাকে।
- কিছু না মিললে ফল একটি ফাঁকা Array, `undefined` বা `null` নয়। তাই ফলের ওপর নিশ্চিন্তে `length` বা `map` ব্যবহার করা যায়।

`map` এবং `filter`-এর পার্থক্য এক বাক্যে: `map` প্রতিটি Element **বদলায়**, সংখ্যা একই থাকে; `filter` কিছু Element **বেছে নেয়**, Element বদলায় না।

৬.৯ find

`find` শর্ত মেলা **প্রথম Element-টি** ফেরত দেয় এবং সেখানেই থেমে যায়। কিছু না মিললে `undefined`।

find.js · JavaScript — চলবে: Browser Console ও Node.js

```
const products = [
  { id: 'P1', name: 'Notebook', price: 120 },
  { id: 'P2', name: 'Pen', price: 15 },
  { id: 'P3', name: 'Bag', price: 950 },
];

const pen = products.find((product) => product.id === 'P2');
const missing = products.find((product) => product.id === 'P9');

console.log(pen.name);
console.log(missing);
console.log(products.findIndex((product) => product.id === 'P3'));
```

Output

```
Pen
undefined
2
```

	filter	find
ফেরত দেয়	মেলা সব Element-এর Array	মেলা প্রথম Element-টি নিজে
কিছু না মিললে	`[]`	`undefined`
ব্যবহার	"যেগুলো..."	"যেটি..." (যেমন id দিয়ে একটি খোঁজা)

`find`-এর ফল ব্যবহারের আগে সেটি `undefined` কি না যাচাই করুন। ওপরে `missing.name` লিখলে Program `TypeError` দিয়ে থেমে যেত। `findIndex()` একই কাজ করে, কিন্তু Element-এর বদলে তার Index দেয়, আর না পেলে `-1`।

৬.১০ reduce

`map` ও `filter` একটি Array থেকে আরেকটি Array তৈরি করে। `reduce` পুরো Array-কে গুটিয়ে **একটি মান** তৈরি করে: যোগফল, সর্বোচ্চ, অথবা অন্য কোনো সারমান।

এটি অধ্যায় ৪-এর Accumulator ধরনটিরই Method-রূপ। Callback দুটি মান নেয়: এ পর্যন্ত জমা ফল (Accumulator) এবং বর্তমান Element; আর ফেরত দেয় নতুন জমা ফল। `reduce`-এর দ্বিতীয় Argument হলো Accumulator-এর শুরুর মান।

reduce.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950];

const total = prices.reduce((sum, price) => sum + price, 0);

console.log(total);
```

Output

```
1085
```

ধাপ	sum (জমা ফল)	price (বর্তমান)	ফেরত দেয়
শুরু	0	—	—
১	0	120	120
২	120	15	135
৩	135	950	1085

প্রতিটি ধাপে Callback যা ফেরত দেয়, পরের ধাপে সেটিই `sum` হয়ে আসে। শেষ ধাপের ফলটি `reduce`-এর ফল।

Object-এর তালিকায় এটি বিশেষ কাজে লাগে।

reduce-objects.js · JavaScript — চলবে: Browser Console ও Node.js

```
const products = [
  { name: 'Notebook', price: 120, stock: 40 },
  { name: 'Bag', price: 950, stock: 5 },
];

const stockValue = products.reduce(
  (sum, product) => sum + product.price * product.stock,
  0,
);

console.log(stockValue);
console.log([].reduce((sum, value) => sum + value, 0));
```

Output

```
9550
0
```

> **মনে রাখুন:** `reduce` -এ শুরুর মান (এখানে `0`) সব সময় দিন। না দিলে প্রথম Element-টিই শুরুর মান হয়ে যায়; Object-এর তালিকায় তাতে ভুল ফল আসে, এবং ফাঁকা Array-তে Program `TypeError` দিয়ে থেমে যায়। শুরুর মান দিলে ফাঁকা Array-তে সেই মানটিই ফেরত আসে, যেমন শেষ লাইনে।

`reduce` শক্তিশালী, কিন্তু সহজে দুর্বোধ্য হয়ে যায়। যোগফল বা একটি সারমানের জন্য এটি ব্যবহার করুন। যে কাজ `map` বা `filter` দিয়ে হয়, তা `reduce` দিয়ে করবেন না।

৬.১১ Mutation বনাম নতুন Array

কোনো কাজ মূল Array-টিকে বদলে ফেললে সেটি **Mutating**; মূলটি অক্ষত রেখে নতুন Array ফেরত দিলে **Non-mutating**।

মূল Array বদলায় (Mutating)	নতুন মান দেয় (Non-mutating)
`push`, `pop`, `shift`, `unshift`	`slice`, `map`, `filter`
`splice`	`find`, `reduce`, `includes`, `indexOf`, `join`
`sort`, `reverse`	`[...array]` (Spread)
Index-এ Assignment (`arr[0] = x`)	`at`

এটি কেন গুরুত্বপূর্ণ? কারণ Array একটি Reference (অধ্যায় ২)। একটি Function-এ Array পাঠালে Function-টি সেই একই Array-র ঠিকানা পায়।

mutation-bug.js · JavaScript চলবে: Node.js

```
function addBonusBad(marks) {
  marks.push(5);
  return marks;
}

const original = [70, 80];
const updated = addBonusBad(original);

console.log(original);
console.log(original === updated);

function addBonusGood(marks) {
  return [...marks, 5];
```

```
}

const safe = [70, 80];
const result = addBonusGood(safe);

console.log(safe);
console.log(result);
```

Output

```
[ 70, 80, 5 ]
true
[ 70, 80 ]
[ 70, 80, 5 ]
```

`addBonusBad` যে তালিকাটি পেয়েছিল, সেটিই বদলে দিয়েছে। যে Code `original` তৈরি করেছিল, সে জানেই না তার তালিকায় একটি নতুন মান ঢুকে গেছে। `addBonusGood` একটি নতুন Array তৈরি করে ফেরত দেয়; মূলটি অক্ষত।

`[...marks]` হলো **Spread**: এটি একটি Array-র Element-গুলোকে নতুন Array-তে ছড়িয়ে দেয়। `[...marks, 5]` মানে "আগের সব Element, তারপর `5`"।

Mutation ছাড়া তিনটি সাধারণ কাজ

কাজ	Mutating উপায়	Non-mutating উপায়
শেষে যোগ	`items.push(x)`	`[...items, x]`
একটি বাদ	`items.splice(i, 1)`	`items.filter(...)`
একটি বদলানো	`items[i] = x`	`items.map(...)`

immutable-update.js · JavaScript চলবে: Node.js

```
const stock = [
  { id: 'P1', count: 40 },
  { id: 'P2', count: 3 },
];

const afterSale = stock.map((item) =>
  item.id === 'P1' ? { ...item, count: item.count - 1 } : item,
);

console.log(afterSale[0].count);
console.log(stock[0].count);
```

Output

```
39
40
```

`map` প্রতিটি Element দেখে: id মিললে `{ ...item, count: ... }` দিয়ে একটি **নতুন Object** তৈরি করে, যাতে পুরোনো সব তথ্য আছে এবং `count` নতুন; না মিললে পুরোনো Object-টিই ফেরত দেয়। মূল `stock` অপরিবর্তিত। এখানে `item.count = ...` লিখলে মূল Object-টিই বদলে যেত, কারণ নতুন Array-র ভেতরেও একই Object-গুলোর ঠিকানা থাকে।

কোনটি ব্যবহার করবেন? নিজের তৈরি, Function-এর ভেতরের একটি স্থানীয় Array-তে `push` করা সম্পূর্ণ ঠিক আছে। কিন্তু Argument হিসেবে পাওয়া Array, অথবা Program-এর একাধিক জায়গা থেকে ব্যবহৃত Array বদলাবেন না; নতুন Array তৈরি করুন।

৬.১২ Data রূপান্তরের ধারা

`map` ও `filter` Array ফেরত দেয়, তাই একটির ফলের ওপর সরাসরি আরেকটি Method Call করা যায়। একে **Method Chaining** বলে। প্রতিটি ধাপ একটি ছোট, স্পষ্ট কাজ করে।

chaining.js · JavaScript — চলবে: Browser Console ও Node.js

```
const products = [
  { name: 'Notebook', price: 120, stock: 40 },
  { name: 'Pen', price: 15, stock: 0 },
  { name: 'School Bag', price: 950, stock: 5 },
  { name: 'Calculator', price: 650, stock: 0 },
];

const availableNames = products
  .filter((product) => product.stock > 0)
  .map((product) => product.name)
  .join(', ');

const availableValue = products
  .filter((product) => product.stock > 0)
  .reduce((sum, product) => sum + product.price * product.stock, 0);

console.log(availableNames);
console.log(availableValue);
```

Output

```
Notebook, School Bag
9550
```

প্রথম ধারাটি পড়ুন ওপর থেকে নিচে: যেগুলোর মজুত আছে সেগুলো **বাছো**, প্রতিটির নাম **নাও**, কমা দিয়ে **জোড়া লাগাও**। প্রতিটি Method নতুন লাইনে লিখলে ধাপগুলো আলাদাভাবে পড়া যায়।

বাস্তব Data প্রক্রিয়াকরণের অধিকাংশ কাজ এই তিন ধাপের কোনো সমন্বয়:

প্রশ্ন	Method
কোনগুলো দরকার?	`filter`
প্রতিটির কোন রূপ দরকার?	`map`
সব মিলিয়ে কী?	`reduce`
নির্দিষ্ট একটি কোথায়?	`find`

সাধারণত `filter` আগে, `map` পরে লিখুন: আগে তালিকা ছোট করে নিলে পরের ধাপে কম Element নিয়ে কাজ করতে হয়।

৬.১৩ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
`map`-এর Callback-এ `return` নেই	`[ undefined, undefined ]`	`{ }` সরান অথবা `return` লিখুন
`filter` আশা করে `map` ব্যবহার	`[ true, false, true ]`	বাছাইয়ের জন্য `filter`
`push()`-এর ফলকে Array ভাবা	একটি সংখ্যা পাওয়া যায়	`push` নতুন `length` ফেরত দেয়
`slice` ও `splice` গুলিয়ে ফেলা	মূল Array অপ্রত্যাশিতভাবে বদলায়	`splice` বদলায়, `slice` বদলায় না
`find`-এর ফল যাচাই ছাড়া ব্যবহার	`TypeError: Cannot read properties of undefined`	আগে `=== undefined` যাচাই করুন
`reduce`-এ শুরুর মান নেই	ফাঁকা Array-তে `TypeError`; Object-এ ভুল ফল	দ্বিতীয় Argument দিন
`=` দিয়ে Array "Copy"	দুটি নাম একই Array দেখায়	`[...array]`
দুটি Array `===` দিয়ে তুলনা	একই মান হলেও `false`	`===` ঠিকানা তুলনা করে; Element ধরে তুলনা করুন
Argument-এর Array-তে `push` বা `sort`	Call-কারীর তালিকা বদলে যায়	নতুন Array তৈরি করুন
`sort()` Callback ছাড়া সংখ্যায়	`[ 100, 40, 9 ]`	`sort((a, b) => a - b)`

শেষ ভুলটি এতই প্রচলিত যে আলাদাভাবে দেখা দরকার।

sort-trap.js · JavaScript চলবে: Node.js

```
const scores = [40, 100, 9];

console.log([...scores].sort());
console.log([...scores].sort((a, b) => a - b));
console.log(scores);
```

Output

```
[ 100, 40, 9 ]
[ 9, 40, 100 ]
[ 40, 100, 9 ]
```

`sort()` Element-গুলোকে String হিসেবে তুলনা করে, তাই `'100'` আসে `'40'`-এর আগে (অধ্যায় ৩-এর String তুলনা)। সংখ্যা সাজাতে একটি তুলনাকারী Callback দিতে হয়। `sort` মূল Array বদলায়, তাই এখানে আগে `[...scores]` দিয়ে Copy করা হয়েছে; শেষ লাইন দেখাচ্ছে `scores` অক্ষত। Sorting নিয়ে বিস্তারিত অধ্যায় ১৮-এ।

৬.১৪ Debugging নির্দেশিকা

Method Chain-এ সমস্যা হলে পুরো ধারাটি একসঙ্গে বোঝার চেষ্টা করবেন না। প্রতিটি ধাপের ফল আলাদা Variable-এ নিয়ে দেখুন।

debug-chain.js · JavaScript চলবে: Node.js

```
const products = [
  { name: 'Notebook', stock: 40 },
  { name: 'Pen', stock: 0 },
];

const available = products.filter((product) => product.stock > 0);
console.log('filter-এর পর:', available.length);

const names = available.map((product) => product.name);
console.log('map-এর পর:', names);

console.log(JSON.stringify(available));
```

Output

```
filter-এর পর: 1
map-এর পর: [ 'Notebook' ]
[{"name":"Notebook","stock":40}]
```

`JSON.stringify()` একটি Object বা Array-কে এক লাইনের String-এ পরিণত করে। বড় বা গভীর Data এক নজরে দেখতে এবং দুটি Array-র বিষয়বস্তু মেলাতে এটি কাজে লাগে।

Array নিয়ে কাজ প্রত্যাশামতো না হলে:

১. Chain ভেঙে প্রতিটি ধাপের ফল ও তার `length` দেখুন।
২. ফলে `undefined` থাকলে দেখুন Callback কিছু `return` করছে কি না।
৩. Callback-এর ভেতরে `console.log()` বসিয়ে দেখুন প্রতিটি Element-এ কী আসছে।
৪. মূল Array অপ্রত্যাশিতভাবে বদলে গেলে খুঁজুন কোথায় `push`, `splice`, `sort` বা Index-এ Assignment হয়েছে।
৫. দুটি Variable একই Array কি না `===` দিয়ে যাচাই করুন।
৬. ফাঁকা Array এবং এক Element-এর Array দিয়ে পরীক্ষা করুন।
৭. `TypeError: ... is not a function` এলে দেখুন যার ওপর Method Call করছেন সেটি আদৌ Array কি না (`Array.isArray()`); হয়তো সেটি `push()`-এর ফেরত দেওয়া সংখ্যা।

৬.১৫ পেশাদার অভ্যাস

- **উদ্দেশ্য অনুযায়ী Method বেছে নিন।** রূপান্তরে `map`, বাছাইয়ে `filter`, একটি খুঁজতে `find`, সারমানে `reduce`, শুধু কিছু করতে `for...of`।
- **Argument হিসেবে পাওয়া Array বদলাবেন না।** নতুন Array ফেরত দিন।
- **`reduce`-এ সব সময় শুরুর মান দিন।**
- **Callback ছোট ও Pure রাখুন।** `map`-এর Callback-এর ভেতরে বাইরের Variable বদলাবেন না।
- **Callback-এর Parameter-এর অর্থবহ নাম দিন:** `product`, `price` — `x` বা `item1` নয়।
- **Chain-এর প্রতিটি ধাপ নতুন লাইনে।** তিন-চার ধাপের বেশি হলে মাঝের ফলকে একটি নাম দিন।
- **একটি Array-তে এক ধরনের মান।**
- **Array-র নাম বহুবচনে।**
- **ফাঁকা তালিকার আচরণ ভেবে নিন।** `filter` ফাঁকা Array দেয়, `find` দেয় `undefined`, শুরুর মানসহ `reduce` দেয় শুরুর মানটি।

৬.১৬ Project: Inventory Dashboard

একটি দোকানের মজুতের তালিকা সামলানোর Program লিখবেন: পণ্য যোগ, বাদ, মজুত বদলানো, খোঁজা, কম মজুতের তালিকা এবং মোট মূল্য। প্রতিটি কাজ মূল তালিকা না বদলে নতুন তালিকা ফেরত দেবে।

Project-এর চাহিদা

- প্রতিটি পণ্য একটি Object: `id`, `name`, `price` (টাকায়, পূর্ণসংখ্যা) এবং `stock`।

- পণ্য যোগ, বাদ এবং মজুত বদলানোর Function মূল Array বা তার Object বদলাবে না; নতুন Array ফেরত দেবে।
- মজুত কখনো শূন্যের নিচে নামবে না।
- id দিয়ে একটি পণ্য খোঁজা যাবে; না পেলে `undefined`।
- মজুত একটি সীমার সমান বা কম হলে পণ্যটি "মজুত কম" তালিকায় আসবে; সীমা না দিলে ৫ ধরা হবে।
- পুরো মজুতের মোট মূল্য (দাম × মজুত-এর যোগফল) দেখাতে হবে।

File-এর কাঠামো

Folder-এর কাঠামো

```
inventory-dashboard/
├── inventory.js
├── README.md
└── tests/
    └── inventory.test.js
```

inventory.js

File-টি তিন ধাপে দেখানো হলো। তিনটি অংশ পরপর একই File-এ লিখুন; `// ...` লাইনগুলো লিখতে হবে না।

ধাপ ১: তালিকা বদলানোর Function

inventory.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```javascript
'use strict';

const LOW_STOCK_LIMIT = 5;

// নতুন পণ্যসহ একটি নতুন Array ফেরত দেয়।
const addProduct = (products, product) => [...products, product];

// নির্দিষ্ট id-র পণ্য বাদ দিয়ে একটি নতুন Array ফেরত দেয়।
const removeProduct = (products, id) =>
  products.filter((product) => product.id !== id);

// id দিয়ে একটি পণ্য খোঁজে; না পেলে undefined।
const findProduct = (products, id) =>
  products.find((product) => product.id === id);

// একটি পণ্যের মজুত বদলে নতুন Array ফেরত দেয়। মজুত শূন্যের নিচে নামে না।
function changeStock(products, id, change) {
  return products.map((product) => {
    if (product.id !== id) {
      return product;
    }

    const next = product.stock + change;
```

```
    return { ...product, stock: next < 0 ? 0 : next };
  });
}

// ...
```

অনুচ্ছেদ ৬.১১-এর তিনটি Non-mutating ধরনই এখানে আছে: যোগ করতে Spread, বাদ দিতে `filter`, বদলাতে `map`। `changeStock`-এর Callback-এ একাধিক ধাপ আছে, তাই `{ }` এবং `return` ব্যবহার করা হয়েছে। যে পণ্যের id মেলে না, সেটি অপরিবর্তিত ফেরত যায়; যেটি মেলে, তার একটি নতুন Object তৈরি হয়।

ধাপ ২: তথ্য বের করার Function

inventory.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// মজুত সীমার সমান বা কম, এমন পণ্যের Array।
const lowStock = (products, limit = LOW_STOCK_LIMIT) =>
  products.filter((product) => product.stock <= limit);

// পুরো মজুতের মোট মূল্য (টাকায়)।
const totalValue = (products) =>
  products.reduce((sum, product) => sum + product.price * product.stock, 0);

// প্রতিটি পণ্যের এক লাইনের বিবরণ।
const describe = (products) =>
  products.map((p) => `${p.id} ${p.name}: ${p.stock} x ${p.price}`);

// ...
```

তিনটি প্রশ্ন, তিনটি Method: কোনগুলো (`filter`), সব মিলিয়ে কত (`reduce`), প্রতিটির কোন রূপ (`map`)। `lowStock`-এ একটি Default Parameter আছে। `describe`-এর Callback-এ লাইন ছোট রাখতে Parameter-এর নাম `p` রাখা হয়েছে; এক লাইনের ছোট Callback-এ এটি গ্রহণযোগ্য।

ধাপ ৩: Dashboard দেখানো

inventory.js · JavaScript নির্বাচিত অংশ (Selected Excerpt) চলবে: Node.js

```
// ...

// পণ্যের তালিকা এবং সারসংক্ষেপ দেখায়।
function printDashboard(products) {
  for (const line of describe(products)) {
    console.log(line);
  }

  const lowNames = lowStock(products).map((product) => product.name);
```

```
  console.log(`পণ্য: ${products.length}টি`);
  console.log(`মোট মূল্য: ${totalValue(products)} টাকা`);
  console.log(`মজুত কম: ${lowNames.join(', ')}`);
}

const initial = [
  { id: 'P1', name: 'Notebook', price: 120, stock: 40 },
  { id: 'P2', name: 'Pen', price: 15, stock: 3 },
  { id: 'P3', name: 'School Bag', price: 950, stock: 5 },
];
const calculator = { id: 'P4', name: 'Calculator', price: 650, stock: 12 };

let products = addProduct(initial, calculator);
products = changeStock(products, 'P1', -38);
products = removeProduct(products, 'P2');

printDashboard(products);
console.log(`শুরুর তালিকায় এখনো ${initial.length}টি পণ্য`);
console.log(`শুরুর তালিকায় Notebook-এর মজুত: ${initial[0].stock}`);
```

Output

```
P1 Notebook: 2 x 120
P3 School Bag: 5 x 950
P4 Calculator: 12 x 650
পণ্য: 3টি
মোট মূল্য: 12790 টাকা
মজুত কম: Notebook, School Bag
শুরুর তালিকায় এখনো 3টি পণ্য
শুরুর তালিকায় Notebook-এর মজুত: 40
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **তিনটি পরিবর্তন, কোনো Mutation নয়।** `addProduct`, `changeStock` ও `removeProduct`—প্রতিটি একটি নতুন Array ফেরত দেয়, যা আবার `products`-এ রাখা হয়েছে। `products` নামটি নতুন Array-র দিকে সরানো হচ্ছে বলে এটি `let`; কোনো Array নিজে বদলাচ্ছে না।
- **প্রমাণ শেষ দুই লাইনে।** তিনটি পরিবর্তনের পরও `initial`-এ তিনটি পণ্য, এবং Notebook-এর মজুত ৪০। `changeStock`-এ `{ ...product, stock: ... }`-এর বদলে `product.stock = ...` লিখলে `initial[0].stock`-ও ২ হয়ে যেত, কারণ দুটি Array একই Object-এর ঠিকানা ধরে থাকত।
- **মজুত ৪০ থেকে ৩৮ কমে ২।** আরও বেশি কমাতে চাইলে Ternary সেটিকে শূন্যে আটকে দিত।
- **সীমার সমান মানও ধরা হয়।** School Bag-এর মজুত ঠিক ৫, এবং শর্তটি `<=`, তাই এটি "মজুত কম" তালিকায় আছে।
- **`for...of` দেখানোর জন্য।** `printDashboard` প্রতিটি লাইন দেখায়; এখানে কোনো মান তৈরি হচ্ছে না, শুধু একটি কাজ হচ্ছে, তাই `map` নয়, `for...of`।

- **Function-এর স্তর।** `printDashboard` ছাড়া সব Function Pure। অধ্যায় ৫-এর Invoice Calculator-এর মতোই, দেখানোর কাজ এক জায়গায়।

`findProduct` এই Demo-তে Call করা হয়নি; এটি Test-এ এবং অনুশীলনী ১০-এ ব্যবহৃত হয়েছে।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
node inventory.js
```

Call	যা পাওয়ার কথা
`addProduct(initial, calculator)`	চারটি পণ্যের নতুন Array; `initial` -এ তিনটিই থাকে
`changeStock(products, 'P1', -38)`	P1-এর মজুত ২; বাকিগুলো অপরিবর্তিত
`changeStock(products, 'P1', -500)`	P1-এর মজুত ০ (ঋণাত্মক হয় না)
`changeStock(products, 'P9', 5)`	কোনো পরিবর্তন নয়; অচেনা id উপেক্ষিত
`removeProduct(products, 'P2')`	P2 ছাড়া নতুন Array
`findProduct(products, 'P9')`	`undefined`
`lowStock(initial)`	Pen (৩) এবং School Bag (৫)
`totalValue([])`	`0`

Companion Source Code-এ এই Project-এর ১৬টি Automated Test আছে, যার কয়েকটি বিশেষভাবে যাচাই করে যে মূল Array ও তার Object বদলায়নি। চালানোর Command `README.md` -তে আছে।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`initial` -এর মজুতও বদলে যাচ্ছে	`changeStock` -এ `product.stock = ...`	`{ ...product, stock: ... }` দিয়ে নতুন Object
`products.map is not a function`	`products` -এ `push()` -এর ফল (একটি সংখ্যা) রাখা হয়েছে	`[...products, product]`
Dashboard-এ `undefined` লাইন	`map` -এর Callback-এ `return` নেই	`{ }` ছাড়া লিখুন, বা `return` দিন
মোট মূল্য `NaN`	`reduce` -এর শুরুর মান নেই, বা কোনো `price` সংখ্যা নয়	দ্বিতীয় Argument `0` ; তথ্য যাচাই করুন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
ঠিক ৫ মজুতের পণ্য তালিকায় নেই	`<=` -এর জায়গায় `<`	`product.stock <= limit`
বাদ দেওয়ার পরও পণ্য থাকছে	ফলটি Variable-এ রাখা হয়নি	`products = removeProduct(products, id);`

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য Program-এর ভেতরে লেখা এবং কোথাও সংরক্ষণ করা হয় না। পণ্যের তথ্য যাচাই করা হয় না, এবং একই id দুবার যোগ করা আটকানো হয় না (অনুশীলনী ১০ দেখুন)। অচেনা id-তে মজুত বদলাতে চাইলে কোনো বার্তা দেওয়া হয় না। দাম পূর্ণ টাকায় ধরা হয়েছে; পয়সাসহ হিসাবে অধ্যায় ৫-এর পদ্ধতি প্রয়োগ করতে হবে।

৬.১৭ অনুশীলনী

প্রাথমিক স্তর

১. পাঁচটি শহরের নাম দিয়ে একটি Array তৈরি করুন। প্রথম, শেষ এবং মাঝের শহরটি দেখান।

২. একটি ফাঁকা Array দিয়ে শুরু করে `push` দিয়ে তিনটি সংখ্যা যোগ করুন, তারপর `pop` দিয়ে একটি বাদ দিন। প্রতিটি ধাপের পর `length` দেখান।

৩. না চালিয়ে বলুন: `['a', 'b', 'c', 'd'].slice(1, 3)` কী দেয়, এবং মূল Array-তে কয়টি Element থাকে?

৪. `for...of` দিয়ে `[10, 20, 30]` -এর প্রতিটি মানের দ্বিগুণ দেখান।

৫. নিচের Code-এ `doubled` -এ `[ 2, 4, 6 ]` আসার কথা, কিন্তু আসছে না। ভুলটি ঠিক করুন।

map-bug.js · JavaScript — চলবে: Node.js

```
const numbers = [1, 2, 3];

const doubled = numbers.map((number) => {
  number * 2;
});

console.log(doubled);
```

মধ্যম স্তর

৬. `[72, 28, 45, 91, 60]` নম্বরগুলো থেকে `filter` দিয়ে উত্তীর্ণদের (৩৩ বা বেশি) বের করুন, এবং `map` দিয়ে প্রতিটির সঙ্গে ৫ নম্বর Bonus যোগ করুন।

৭. `reduce` দিয়ে `[120, 15, 950, 280]` -এর যোগফল এবং গড় বের করুন।

৮. শিক্ষার্থীদের একটি Array থেকে (`name` ও `mark` সহ Object) `find` দিয়ে প্রথম সেই শিক্ষার্থীকে খুঁজুন, যার নম্বর ৯০ বা বেশি। না পাওয়া গেলে `কেউ নেই` দেখান।

৯. `withoutFirst` নামে একটি Function লিখুন, যা একটি Array-র প্রথম Element বাদ দিয়ে নতুন Array ফেরত দেয়, কিন্তু মূল Array বদলায় না।

চ্যালেঞ্জ স্তর

১০. **নকল id আটকানো:** `addUnique` Function লিখুন, যা পণ্য যোগ করে, কিন্তু একই id আগে থেকে থাকলে মূল Array-টিই অপরিবর্তিত ফেরত দেয়।

১১. **দামি পণ্যের নাম:** পণ্যের Array থেকে যেগুলোর মোট মূল্য (দাম × মজুত) ৪০০০ টাকা বা বেশি, সেগুলোর নাম কমা দিয়ে আলাদা করে দেখান। `filter`, `map` ও `join` ব্যবহার করুন।

১২. **দাম বাড়ানো:** `raisePrices` Function লিখুন, যা সব পণ্যের দাম একটি নির্দিষ্ট শতাংশ বাড়িয়ে নতুন Array ফেরত দেয়। মূল Array এবং তার Object যেন না বদলায়, তা দেখান।

৬.১৮ অনুশীলনীর সমাধান

সমাধান ১

cities.js · JavaScript — চলবে: Browser Console ও Node.js

```
const cities = ['Dhaka', 'Bogura', 'Sylhet', 'Khulna', 'Rajshahi'];

console.log(cities[0]);
console.log(cities.at(-1));
console.log(cities[2]);
```

Output

```
Dhaka
Rajshahi
Sylhet
```

সমাধান ২

push-three.js · JavaScript — চলবে: Browser Console ও Node.js

```
const numbers = [];

numbers.push(10);
console.log(numbers.length);

numbers.push(20);
numbers.push(30);
console.log(numbers.length);

numbers.pop();
console.log(numbers.length);
```

Output

```
1
3
2
```

সমাধান ৩ — `[ 'b', 'c' ]`: Index ১ ও ২, কারণ শেষ Index অন্তর্ভুক্ত নয়। মূল Array-তে চারটি Element-ই থাকে, কারণ `slice` মূল Array বদলায় না।

সমাধান ৪

double-each.js · JavaScript — চলবে: Browser Console ও Node.js

```
for (const value of [10, 20, 30]) {
  console.log(value * 2);
}
```

Output

```
20
40
60
```

সমাধান ৫ — Callback-এ `{ }` আছে কিন্তু `return` নেই, তাই প্রতিটি Element-এর জন্য `undefined` ফেরত যাচ্ছে। `{ }` সরিয়ে দিলে Expression-টির মান নিজে থেকেই ফেরত যায়।

map-fixed.js · JavaScript — চলবে: Node.js

```
const numbers = [1, 2, 3];

const doubled = numbers.map((number) => number * 2);

console.log(doubled);
```

Output

```
[ 2, 4, 6 ]
```

সমাধান ৬

passed-bonus.js · JavaScript — চলবে: Node.js

```
const marks = [72, 28, 45, 91, 60];

const result = marks
  .filter((mark) => mark >= 33)
  .map((mark) => mark + 5);

console.log(result);
```

Output

```
[ 77, 50, 96, 65 ]
```

সমাধান ৭

sum-average.js · JavaScript — চলবে: Browser Console ও Node.js

```
const prices = [120, 15, 950, 280];

const sum = prices.reduce((total, price) => total + price, 0);

console.log(sum);
console.log(sum / prices.length);
```

Output

```
1365
341.25
```

সমাধান ৮

top-student.js · JavaScript — চলবে: Browser Console ও Node.js

```
const students = [
  { name: 'Sara', mark: 80 },
  { name: 'Mina', mark: 93 },
  { name: 'Rafi', mark: 95 },
];

const top = students.find((student) => student.mark >= 90);

console.log(top === undefined ? 'কেউ নেই' : top.name);
```

Output

```
Mina
```

Rafi-র নম্বর বেশি, কিন্তু `find` প্রথম যাকে পায় তাকেই ফেরত দেয় এবং সেখানে থেমে যায়।

সমাধান ৯

without-first.js · JavaScript — চলবে: Node.js

```
const withoutFirst = (items) => items.slice(1);

const queue = ['Sara', 'Rafi', 'Mina'];

console.log(withoutFirst(queue));
console.log(queue);
```

Output

```
[ 'Rafi', 'Mina' ]
[ 'Sara', 'Rafi', 'Mina' ]
```

`shift()` একই Element বাদ দিত, কিন্তু মূল Array বদলে ফেলত।

সমাধান ১০

add-unique.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
function addUnique(products, product) {
  const existing = products.find((item) => item.id === product.id);

  if (existing !== undefined) {
    return products;
  }

  return [...products, product];
}

const products = [{ id: 'P1', name: 'Notebook' }];
const added = addUnique(products, { id: 'P2', name: 'Pen' });
const duplicate = addUnique(added, { id: 'P2', name: 'Pencil' });

console.log(added.length);
console.log(duplicate.length);
console.log(duplicate === added);
```

Output

```
2
2
true
```

সমাধান ১১

valuable.js · JavaScript — চলবে: Browser Console ও Node.js

```javascript
const products = [
  { name: 'Notebook', price: 120, stock: 40 },
  { name: 'Pen', price: 15, stock: 3 },
  { name: 'School Bag', price: 950, stock: 5 },
];

const valuable = products
  .filter((product) => product.price * product.stock >= 4000)
  .map((product) => product.name)
  .join(', ');

console.log(valuable);
```

Output

```
Notebook, School Bag
```

সমাধান ১২

raise-prices.js · JavaScript চলবে: Browser Console ও Node.js

```
function raisePrices(products, percent) {
  return products.map((product) => ({
    ...product,
    price: product.price + Math.round(product.price * percent / 100),
  }));
}

const products = [
  { name: 'Notebook', price: 120 },
  { name: 'Pen', price: 15 },
];
const raised = raisePrices(products, 10);

console.log(raised[0].price);
console.log(raised[1].price);
console.log(products[0].price);
```

Output

```
132
17
120
```

Arrow Function থেকে সরাসরি একটি Object ফেরত দিতে হলে Object-টিকে প্রথম বন্ধনীতে ঘিরতে হয়: `=> ({ ... })`। নইলে JavaScript `{ }`-কে Function-এর Body ভাবে।

৬.১৯ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. `const a = ['x', 'y', 'z'];` হলে `a.at(-1)` কী দেয়?

ক) `'x'`

খ) `undefined`

গ) `'z'`

ঘ) একটি Error

প্রশ্ন ২. `push()` কী ফেরত দেয়?

ক) Array-র নতুন `length`

খ) নতুন Array

গ) যোগ করা Element

ঘ) `undefined`

প্রশ্ন ৩. কোনটি Array-র শুরু থেকে একটি Element বাদ দেয়?

ক) `pop()`

খ) `unshift()`

গ) `push()`

ঘ) `shift()`

প্রশ্ন ৪. `slice` এবং `splice`-এর মূল পার্থক্য কী?

ক) `slice` শুধু String-এ কাজ করে

খ) `splice` মূল Array বদলায়; `slice` বদলায় না

গ) `slice` মূল Array বদলায়; `splice` বদলায় না

ঘ) কোনো পার্থক্য নেই

প্রশ্ন ৫. `[1, 2, 3].map((n) => n * 2)`-এর ফল কী?

ক) `6`

খ) `[ 1, 2, 3 ]`

গ) `[ 2, 4, 6 ]`

ঘ) `[ true, true, true ]`

প্রশ্ন ৬. `filter`-এ কোনো Element শর্ত না মিললে কী ফেরত আসে?

ক) `undefined`

খ) `null`

গ) `false`

ঘ) একটি ফাঁকা Array

প্রশ্ন ৭. `find` কী ফেরত দেয়?

ক) শর্ত মেলা প্রথম Element

খ) শর্ত মেলা সব Element-এর Array

গ) শর্ত মেলা Element-এর সংখ্যা

ঘ) `true` বা `false`

প্রশ্ন ৮. `[10, 20, 30].reduce((sum, n) => sum + n, 0)`-এর ফল কী?

ক) `[ 10, 30, 60 ]`

খ) `60`

গ) `30`

ঘ) `0`

প্রশ্ন ৯. নিচের কোনটি মূল Array বদলায় **না**?

ক) `sort()`

খ) `splice()`

গ) `filter()`

ঘ) `push()`

প্রশ্ন ১০. `const b = a;` লেখার পর `b.push(1)` করলে `a`-র কী হয় (`a` একটি Array)?

ক) কিছু হয় না; `b` একটি আলাদা Copy

খ) `a`-তেও `1` যোগ হয়, কারণ দুটি নাম একই Array দেখায়

গ) একটি Error হয়

ঘ) `a` ফাঁকা হয়ে যায়

৬.২০ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–গ · ২–ক · ৩–ঘ · ৪–খ · ৫–গ · ৬–ঘ · ৭–ক · ৮–খ · ৯–গ · ১০–খ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	গ	`at()`-এ ঋণাত্মক সংখ্যা শেষ থেকে গোনে; `-1` মানে শেষ Element।
২	ক	`push` মূল Array বদলায় এবং নতুন `length` দেয়; নতুন Array নয়।
৩	ঘ	`shift` শুরু থেকে বাদ দেয়; `pop` শেষ থেকে। `unshift` ও `push` যোগ করে।
৪	খ	`slice` একটি অংশের Copy দেয়; `splice` মূল Array কেটে বদলায়।
৫	গ	`map` প্রতিটি Element রূপান্তর করে একই দৈর্ঘ্যের নতুন Array দেয়।
৬	ঘ	`filter` সব সময় একটি Array দেয়; কিছু না মিললে সেটি ফাঁকা।
৭	ক	`find` প্রথম মিলেই থেমে যায়। সবগুলো পেতে `filter`। না পেলে `undefined`।
৮	খ	শুরুর মান `0`; তারপর ১০, ৩০, ৬০। `reduce` একটি মান দেয়, Array নয়।
৯	গ	`filter` নতুন Array দেয়। বাকি তিনটি মূল Array বদলায়।

প্রশ্ন	উত্তর	ব্যাখ্যা
১০	খ	`=` ঠিকানা Copy করে, Array নয়। আলাদা Copy-র জন্য `[...a]`।

৬.২১ অধ্যায়ের সারসংক্ষেপ

- Array ক্রমানুসারে মান রাখে; Index শূন্য থেকে; শেষ Element `at(-1)`।
- `push` / `pop` শেষ প্রান্তে, `unshift` / `shift` শুরুর প্রান্তে যোগ ও বাদ দেয়; চারটিই মূল Array বদলায়।
- `slice` একটি অংশের Copy দেয়; `splice` মূল Array কেটে বদলায়।
- শুধু মান দরকার হলে `for...of`; Index দরকার হলে `for`।
- `map` প্রতিটি Element রূপান্তর করে; দৈর্ঘ্য একই থাকে।
- `filter` শর্ত মেলা Element বেছে নেয়; না মিললে ফাঁকা Array।
- `find` প্রথম মিলটি দেয়; না মিললে `undefined`।
- `reduce` পুরো Array-কে একটি মানে গুটিয়ে আনে; শুরুর মান সব সময় দিন।
- Mutating Method মূল Array বদলায়; Non-mutating Method নতুন মান দেয়। Argument হিসেবে পাওয়া Array বদলাবেন না।
- Mutation ছাড়া: যোগ করতে `[...items, x]`, বাদ দিতে `filter`, বদলাতে `map` ও `{ ...item }`।
- `filter`, `map`, `reduce` জুড়ে Data রূপান্তরের ধারা তৈরি হয়।

পরের অধ্যায়ে Object: Property, Method, Destructuring এবং বাস্তব Data-র গঠন—যা এই অধ্যায়ের Object-এর তালিকাগুলোকে আরও শক্তিশালী করবে।

৬.২২ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Array Literal	`[ ]` দিয়ে লেখা Array।
Element / Index	Array-র একটি মান / তার অবস্থানের নম্বর।
Method	একটি মানের সঙ্গে যুক্ত Function, যেমন `array.push()`।
Mutation	একটি বিদ্যমান Array বা Object-কে বদলে ফেলা।
Mutating / Non-mutating	যে Method মূলটি বদলায় / যেটি নতুন মান দেয়।
Spread	`...`: একটি Array বা Object-এর বিষয়বস্তু নতুনটিতে ছড়িয়ে দেওয়া।
`map`	প্রতিটি Element রূপান্তর করে নতুন Array।
`filter`	শর্ত মেলা Element নিয়ে নতুন Array।

পরিভাষা	অর্থ
`find`	শর্ত মেলা প্রথম Element।
`reduce`	পুরো Array থেকে একটি মান।
Accumulator	`reduce`-এ ধাপে ধাপে জমা হওয়া ফল।
Method Chaining	একটি Method-এর ফলের ওপর সরাসরি আরেকটি Call করা।

৬.২৩ আরও পড়ুন

- **MDN — Array:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array` — সব Array Method-এর তালিকা।
- **MDN — Indexed collections:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Indexed_collections` — Array-র নির্দেশিকা।
- **MDN — Array.prototype.map():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/map` — `map`-এর বিবরণ।
- **MDN — Array.prototype.reduce():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/reduce` — `reduce`-এর বিবরণ ও উদাহরণ।
- **MDN — for...of:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/for...of` — `for...of` Statement।